

Na závěrečný projekt nepotřebuješ nic nového – jen dát dohromady to, co už umíš. Zkontroluj si rozklad problému, větvení s kontrolou vstupu, funkce a testy; pro mini-projekt 1. ročníku i vážený průměr.

1. Průměr a vážený průměr **ZŠ**

Kde to najdeš: Začátečníci H23 Tabulky, diagramy a průměr, 1.+2. H55 Statistické charakteristiky

Obyčejný průměr sečte hodnoty a vydělí jejich počtem. Když mají hodnoty různou důležitost nebo četnost (kilogramy, hodiny, váhy), použij vážený průměr: každou hodnotu vynásob její vahou, součiny sečti a vyděl součtem vah. Právě tohle počítá funkce `vazeny_prumer` z mini-projektu.

$$\bar{x} = \frac{w_1x_1 + w_2x_2 + \dots + w_nx_n}{w_1 + w_2 + \dots + w_n} \text{ vážený průměr}$$

Příklad: Obchod prodal 3 kg jablek po 30 Kč, 1 kg po 40 Kč a 2 kg po 25 Kč. Jaká byla průměrná cena za kilogram?

1. Součiny: $3 \cdot 30 + 1 \cdot 40 + 2 \cdot 25 = 90 + 40 + 50 = 180$ Kč.
2. Součet vah: $3 + 1 + 2 = 6$ kg.
3. Průměr: $\frac{180}{6} = 30$ Kč/kg.

Zkus sám:

1. Auto jelo 2 h rychlostí 60 km/h a 1 h rychlostí 90 km/h. Jaká byla průměrná rychlost?
2. Hodnoty 1, 1, 5 mají váhy 1, 1, 2. Jaký je vážený průměr?

2. Rozklad problému a algoritmus **SS**

Kde to najdeš: 1.+2. H4 Co je algoritmus, 1.+2. H6 Zápis algoritmu

Velký problém rozlož na menší části, z nichž každou zvládneš samostatně – každá pak může být jedna funkce. Pro každou část si ujasni vstup, výstup a postup. Zapiš ji nejdřív slovně nebo pseudokódem, teprve potom programuj. Příliš velké zadání raději zmenši – dotažený menší program je lepší než nedokončený velký.

Příklad: Rozlož program „evidence výdajů“ na části.

1. Načtení výdaje (částka a kategorie) s kontrolou vstupu.
2. Uložení do seznamu.
3. Výpočet součtu a průměru, případně podle kategorií.
4. Přehledný výpis výsledků.

Zkus sám:

1. Rozlož hru „hádání čísla“ na části.
2. Proč psát nejdřív pseudokód, a ne rovnou program?

3. Větvení a kontrola vstupu **SS**

Kde to najdeš: 1.+2. H44 Větvení: řešič rovníc

Uživatel může napsat cokoli – číslo, text, prázdný řádek. Program proto vstup nejdřív zkontroluje a teprve pak s ním počítá. Metoda `isdigit()` zjistí, zda text obsahuje jen číslice; `int()` na textu jako „7 h“ skončí chybou `ValueError`. Větvení `if` – `elif` – `else` pak rozhodne, co dělat s platným a co s neplatným vstupem.

Příklad: Program se ptá na věk a uživatel napíše „patnáct“. Jak to ošetříš?

1. Načti vstup jako text: `text = input("Věk: ")`.
2. `if text.isdigit(): vek = int(text)` – jinak vypiš „Zadej věk číslicemi“ a zeptej se znovu.
3. Zkontroluj i rozsah, např. 0 až 120.

Zkus sám:

1. Co vrátí `"12".isdigit()` a co `"-5".isdigit()`?
2. Co se stane při `int("7 h")`?

4. Funkce a dokumentace SS

Kde to najdeš: 1.+2. H52 Podprogramy

Každou část programu zapiš jako funkci s výstižným názvem, parametry a návratovou hodnotou (return). Na první řádek funkce patří docstring – krátký text ve trojitých uvozovkách, co funkce dělá, co dostane a co vrátí. Program pak čte jako obsah knihy: hlavní část jen volá funkce. Soubor README popíše účel, spuštění, příklad a omezení celého programu.

Příklad: Napiš funkci, která vrátí obsah obdélníku, a použij ji pro strany 3 a 4.

1. def obsah_obdelniku(a, b):
2. """Vrátí obsah obdélníku se stranami a, b."""
3. return a * b
4. obsah_obdelniku(3, 4) vrátí 12.

Zkus sám:

1. Co vrátí funkce def je_sude(n): return n % 2 == 0 pro je_sude(7)?
2. Co má obsahovat soubor README?

5. Testy a ladění SS

Kde to najdeš: 1.+2. H47 Testování a ladění

Test porovná skutečný výsledek funkce s očekávaným, v Pythonu nejjednodušeji příkazem assert. Dobrá testovací sada obsahuje běžné případy, hraniční případy (0, prázdný seznam, největší hodnota) i nesmyslné vstupy. Když test selže, najdi chybu: vypiš mezivýsledky nebo projdi program krok za krokem, oprav ji a spusť znovu všechny testy.

Příklad: Navrhni testy pro funkci vazeny_prumer(hodnoty, vahy).

1. Běžný případ: assert vazeny_prumer([1, 5], [1, 1]) == 3.
2. Jedna hodnota: assert vazeny_prumer([4], [2]) == 4.
3. Různé váhy: assert vazeny_prumer([1, 1, 5], [1, 1, 2]) == 3.
4. Hraniční případ: prázdné seznamy – funkce má ohlásit srozumitelnou chybu, ne dělit nulou.

Zkus sám:

1. Navrhni tři testy pro funkci je_sude(n).
2. Proč spouštět všechny testy znovu po každé opravě?

Kde se to doučit

- **Nauč se Python – Testování** – Automatické testy assert a pytest.
<https://naucese.python.cz/course/pyladies/beginners/testing/>
- **Umíme informatiku – Vývoj programů** – Postup vývoje programu, ladění a hledání chyb.
<https://www.umimeinformatiku.cz/cviceni-vyvoj-programu>
- **Choose a License** – Průvodce výběrem licence pro vlastní program (anglicky).
<https://choosealicense.com/>

Výsledky ke kontrole

- **1. Průměr a vážený průměr:** 1) 70 km/h; 2) 3
- **2. Rozklad problému a algoritmus:** 1) vylosovat číslo; opakovat: načíst tip, porovnat, napsat „větší/menší“; po uhodnutí vypsát počet pokusů; 2) oddělíš myšlenku postupu od zápisu v jazyce, dřív najdeš chybu v logice
- **3. Větvení a kontrola vstupu:** 1) True a False; 2) chyba ValueError – text nejde převést na celé číslo
- **4. Funkce a dokumentace:** 1) False; 2) účel programu, jak ho spustit, příklad použití, omezení (a autora a licenci)
- **5. Testy a ladění:** 1) např. je_sude(4) vrátí True, je_sude(7) vrátí False, je_sude(0) vrátí True; 2) oprava jedné chyby mohla rozbít jinou část programu